INSTITUTO MAUÁ DE TECNOLOGIA

ECM306 | Tópicos Avançados em Estrutura de Dados

Remoção de fundo de imagens por corte mínimo em grafos

Amanda B. Ingham
24.01621-7

Ruy D. Monteiro
24.00846-0

12 de novembro de 2026

Sumário

1 Introdução 2
2 Grafos e suas aplicações 4
2.1 Conceitos 4
2.2 Aplicações 4
2.3 Redes de fluxo e corte mínimo 5
3 Modelagem do problema 7
3.1 Passo 1: a foto vira um grafo 7
3.2 Passo 2: peso entre pixels vizinhos 7
3.3 Passo 3: os riscos do usuário 8
3.4 Passo 4: os demais pixels 8
3.5 Passo 5: o corte mínimo é o recorte 9
4 Implementação 10
4.1 Arquitetura 10
4.2 Representação do grafo 11
4.3 Algoritmo de fluxo máximo 11
4.4 Decisões de projeto 11
4.5 Testes 12
4.6 Uso 12
5 Resultados 14
5.1 Desempenho 14
5.2 Efeito dos parâmetros 14
6 Conclusão 16
Referências 17

1 Introdução

Este relatório apresenta o trabalho semestral da disciplina ECM306, dividido em duas partes: uma pesquisa sobre grafos e suas aplicações e uma aplicação em Java que resolve um problema com grafo.

O problema escolhido foi a remoção do fundo de uma fotografia. Ferramentas comerciais fazem isso com redes neurais treinadas em milhões de imagens. Aqui o mesmo resultado é obtido só com estruturas de dados: a foto é transformada em um grafo, e separar o objeto do fundo passa a ser o problema clássico de encontrar o corte mínimo de uma rede de fluxo, na formulação de Boykov e Jolly (2001).

O programa resultante, chamado Thanos, recebe uma foto e dois riscos feitos pelo usuário, um sobre o objeto e outro sobre o fundo, e devolve o objeto recortado em um arquivo PNG transparente. A Figura 1 mostra a entrada e a saída.

Os objetivos do trabalho foram:

1. estudar os conceitos de grafos necessários para o problema, em especial redes de fluxo;
2. modelar a remoção de fundo como um problema de corte mínimo;
3. implementar a solução em Java sem usar bibliotecas de grafos ou de visão computacional;

Figura 1 - Foto original, riscos do usuário e recorte produzido



Fonte: Os autores (2026), sobre foto de Super Festivals, licença CC BY 2.0.

4. organizar o código de forma que o algoritmo possa ser lido e testado isoladamente.

O código-fonte está disponível em Monteiro e Ingham (2026).

2 Grafos e suas aplicações

2.1 Conceitos

Um grafo $G = (V, E)$ é formado por um conjunto V de vértices e um conjunto E de arestas, em que cada aresta liga dois vértices. Quando as arestas têm sentido, o grafo é dirigido. Quando cada aresta carrega um número, o grafo é ponderado, e esse número é o peso da aresta (Cormen et al., 2012).

As duas representações usuais em memória são a matriz de adjacência, que ocupa espaço $O(V^2)$ e responde em tempo constante se dois vértices são vizinhos, e a lista de adjacência, que ocupa $O(V + E)$ e é a escolha natural para grafos esparsos (Sedgewick; Wayne, 2011). Um grafo de pixels é esparso, pois cada pixel tem no máximo quatro vizinhos, de modo que este trabalho usa uma variação da lista de adjacência.

Os dois percursos básicos são a busca em largura (BFS), que visita os vértices em ordem de distância à origem, e a busca em profundidade (DFS), que segue um caminho até o fim antes de retroceder. Ambos custam $O(V + E)$ e são os blocos de construção do algoritmo usado na aplicação.

2.2 Aplicações

Grafos modelam qualquer situação em que importam as relações entre elementos. A Tabela 1 resume as aplicações clássicas estudadas na pesquisa.

Tabela 1 - Aplicações clássicas de grafos

Problema	**Modelo em grafo**	**Algoritmo**
Rota mais curta	Vértices são locais, pesos são distâncias	Dijkstra
Rede de menor custo	Árvore geradora mínima	Kruskal, Prim
Ordem de tarefas	Grafo dirigido acíclico de dependências	Ordenação topológica
Distribuição de tarefas	Emparelhamento em grafo bipartido	Fluxo máximo
Capacidade de uma rede	Rede de fluxo com fonte e sumidouro	Edmonds-Karp, Dinitz
Segmentação de imagens	Pixels são vértices, corte separa regiões	Corte mínimo

Fonte: Os autores (2026), com base em Cormen et al. (2012) e Kleinberg e Tardos (2006).

As duas últimas linhas são a mesma técnica. A segmentação de imagens é apresentada por Kleinberg e Tardos (2006) como aplicação direta de fluxo em redes, e foi o caminho seguido neste trabalho por fugir dos exemplos de mapas e rotas.

2.3 Redes de fluxo e corte mínimo

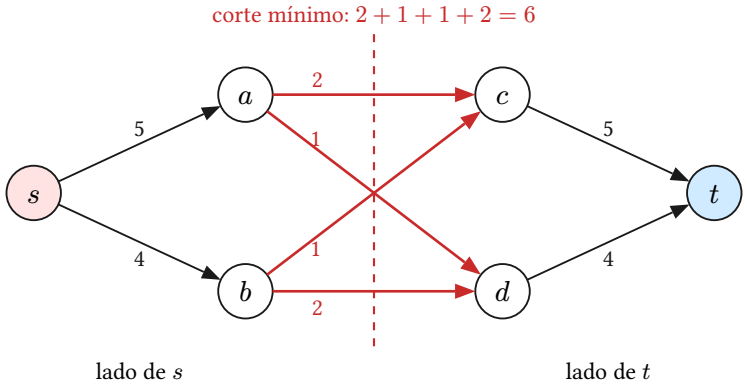
Uma rede de fluxo é um grafo dirigido em que cada aresta (u, v) tem uma capacidade $c(u, v) \geq 0$, com dois vértices especiais: a fonte s e o sumidouro t. Um fluxo atribui a cada aresta um valor que não excede a capacidade e que se conserva em todo vértice intermediário. O problema do fluxo máximo pede o maior valor total que pode sair de s e chegar a t.

Um corte (S, T) divide os vértices em dois grupos, com s em S e t em T. A capacidade do corte é a soma das capacidades das arestas que saem de S e entram em T. O problema do corte mínimo pede o corte de menor capacidade.

Os dois problemas são equivalentes. O teorema do fluxo máximo e corte mínimo, de Ford e Fulkerson (1956), afirma que o valor do fluxo máximo é igual à capacidade do corte mínimo. A Figura 2 mostra uma rede em que ambos valem 6: as quatro arestas centrais ficam saturadas e formam o corte.

A consequência prática é que não é preciso examinar os $2^{|V|}$ cortes possíveis. Basta calcular o fluxo máximo e, ao final, fazer uma busca a partir de s usando só arestas não saturadas: os vértices alcançados formam o lado S do corte mínimo (Cormen et al., 2012).

Figura 2 - Rede de fluxo com as capacidades e o corte mínimo



Fonte: Os autores (2026).

Tabela 2 - Algoritmos de fluxo máximo

Algoritmo	**Escolha dos caminhos**	**Pior caso**
Ford-Fulkerson	Qualquer caminho com capacidade disponível	$O(E \cdot f)$
Edmonds-Karp	O caminho mais curto, por BFS	$O(VE^2)$
Dinitz	Todos os caminhos mais curtos de uma só vez	$O(V^2E)$

Fonte: Ford e Fulkerson (1956), Edmonds e Karp (1972) e Dinic (1970). f é o valor do fluxo máximo.

A Tabela 2 compara os algoritmos estudados. Todos repetem a mesma ideia de encontrar um caminho de s a t com capacidade disponível e enviar fluxo por ele. A diferença está em como os caminhos são escolhidos.

O algoritmo de Dinitz, publicado em 1970 e conhecido na literatura como algoritmo de Dinic (Dinitz, 2006), foi o adotado. Ele trabalha em fases, e cada fase tem dois passos:

1. uma BFS a partir de s atribui a cada vértice um nível, que é a sua distância até a fonte;
2. uma DFS envia fluxo apenas por arestas que vão de um nível para o seguinte, até que nenhum caminho reste. O resultado é chamado de fluxo bloqueante.

As fases se repetem enquanto a BFS alcançar t. Como a distância de s a t aumenta a cada fase, há no máximo $|V| - 1$ fases.

3 Modelagem do problema

A modelagem segue Boykov e Jolly (2001) e é descrita a seguir em cinco passos.

3.1 Passo 1: a foto vira um grafo

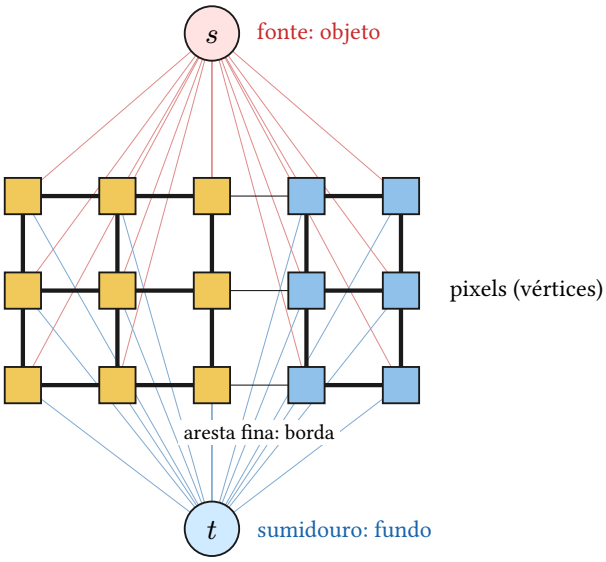
Cada pixel é um vértice. Cada pixel é ligado por uma aresta aos vizinhos de cima, de baixo, da esquerda e da direita. Acrescentam-se dois vértices que não são pixels: a fonte s, que representa o objeto, e o sumidouro t, que representa o fundo. Todo pixel é ligado a ambos. A Figura 3 mostra o grafo de uma imagem de 5 por 3 pixels.

3.2 Passo 2: peso entre pixels vizinhos

A aresta entre dois pixels vizinhos p e q recebe um peso que mede o quanto eles se parecem:

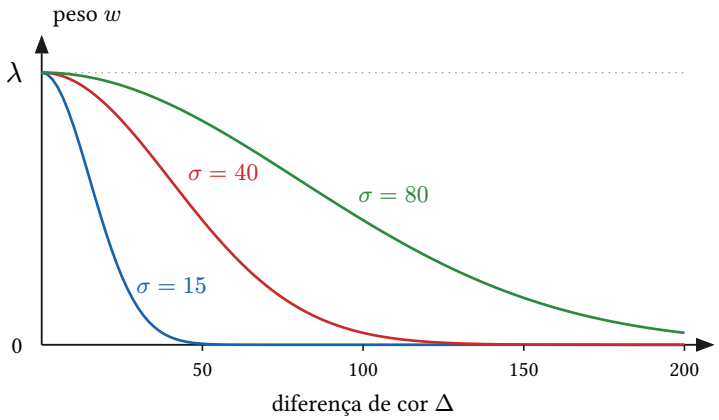
$$w(p,\, q) = \lambda \cdot e^{-\frac{\Delta^2}{2\sigma^2}}, \quad \text{com} \quad \Delta^2 = \left(R_p - R_q\right)^2 + \left(G_p - G_q\right)^2 + \left(B_p - B_q\right)^2 \qquad (1)$$

Figura 3 - Grafo de uma imagem pequena, com a fonte e o sumidouro



Fonte: Os autores (2026).

Figura 4 - Peso da aresta em função da diferença de cor, para três valores de σ



Fonte: Os autores (2026).

em que Δ é a distância entre as duas cores no espaço RGB. Pixels de cores iguais recebem o peso máximo λ. Quanto mais diferentes as cores, menor o peso, como mostra a Figura 4. O resultado é que as arestas ficam fracas exatamente nas bordas dos objetos.

A curva não é uma reta por dois motivos. Diferenças pequenas vêm de ruído e de iluminação, não de bordas, e por isso quase não reduzem o peso. Diferenças grandes já indicam uma borda, e a partir daí o peso se mantém próximo de zero. Os dois parâmetros são ajustáveis: λ controla a altura da curva e σ, a largura.

3.3 Passo 3: os riscos do usuário

O usuário risca alguns pixels do objeto e alguns do fundo. Um pixel riscado como objeto recebe uma aresta de capacidade praticamente infinita com s, e um pixel riscado como fundo, com t. Essas arestas nunca são cortadas, o que fixa esses pixels no lado correto.

3.4 Passo 4: os demais pixels

Para os pixels sem risco, as ligações com s e t vêm da cor. Com os pixels riscados constroem-se dois histogramas de cores, um do objeto e um do fundo, com 16 faixas por canal. A capacidade das arestas de um pixel p de cor c_p é:

$$w(s,\, p) = -\ln P\big(c_p \mid \text{fundo}\big), \qquad w(p,\, t) = -\ln P\big(c_p \mid \text{objeto}\big) \tag{2}$$

Figura 5 - Riscos do usuário e estimativa por cor: vermelho indica objeto, azul indica fundo



Fonte: Os autores (2026).

Um pixel cuja cor é comum no objeto tem probabilidade alta de ser objeto, logo a sua aresta com t é fraca e barata de cortar, enquanto a aresta com s é forte. A Figura 5 mostra essa estimativa para a foto de exemplo. Sozinha ela erra: partes da armadura têm a cor do fundo.

3.5 Passo 5: o corte mínimo é o recorte

Qualquer corte dessa rede separa os pixels em dois grupos, os que continuam ligados a s e os que continuam ligados a t. A capacidade do corte soma duas parcelas: o custo de contrariar a estimativa por cor de cada pixel e o custo de separar pixels vizinhos parecidos.

$$E = \sum_{p} D_p + \sum_{(p,\,q)} w(p,\,q) \cdot \left[L_p \neq L_q\right] \tag{3}$$

em que D_p é a aresta terminal cortada do pixel p e L_p é o lado em que ele ficou. O corte mínimo equilibra as duas parcelas: um pixel com cor de fundo, mas cercado por pixels do objeto, permanece no objeto porque separá-lo dos vizinhos custaria mais. O lado de s no corte mínimo é o objeto recortado.

4 Implementação

A aplicação foi escrita em Java 17, sem ferramenta de build, e compilada diretamente com `javac`. O projeto tem 34 arquivos e 2.315 linhas de código, das quais 1.178 formam o motor.

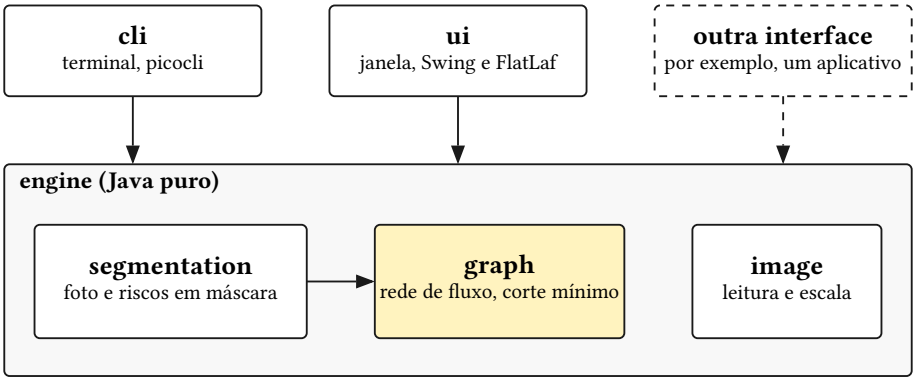
4.1 Arquitetura

O código está dividido em um motor e duas interfaces que o consomem, como mostra a Figura 6. O motor não usa nenhuma biblioteca externa, pois construí-lo era o objetivo do trabalho. As bibliotecas aparecem só nas interfaces: picocli para interpretar a linha de comando e FlatLaf para a aparência da janela Swing.

O motor tem três pacotes. O pacote `graph` contém a rede de fluxo e os algoritmos de fluxo máximo, e não sabe o que é uma imagem: recebe vértices, arestas e capacidades e devolve um corte. O pacote `segmentation` conhece imagens: monta a rede a partir dos pixels e dos riscos e converte o corte em uma máscara. O pacote `image` lê, grava e redimensiona imagens.

As dependências apontam sempre para o motor. O terminal não conhece a janela, e nenhuma interface conhece o pacote `graph`. Um teste automático lê as importações de todos os arquivos e falha se essa regra for violada. Com essa separação, uma nova interface, como um aplicativo para celular, usaria o mesmo motor sem alterá-lo.

Figura 6 - Pacotes da aplicação e suas dependências



Fonte: Os autores (2026).

4.2 Representação do grafo

A rede de fluxo é armazenada em quatro vetores de inteiros, em uma lista de adjacência compacta: `firstEdge` guarda a primeira aresta de cada vértice, `nextEdge` encadeia as arestas de um mesmo vértice, `target` guarda o destino de cada aresta e `capacity`, a capacidade residual. Essa forma evita criar um objeto por aresta, o que importa em um grafo com centenas de milhares delas.

Cada aresta é inserida junto com a sua aresta reversa, em posições consecutivas. Assim, a reversa da aresta `e` é sempre `e ^ 1`, e enviar fluxo se resume a subtrair da capacidade de uma e somar à da outra.

As capacidades são inteiras. Os pesos das equações 1 e 2 são multiplicados por 100 e arredondados, o que elimina erros de ponto flutuante na comparação de capacidades, e o fluxo total é acumulado em `long`.

4.3 Algoritmo de fluxo máximo

Os algoritmos implementam uma mesma interface, `MaxFlowSolver`, com um único método que recebe a rede, a fonte e o sumidouro e devolve o corte mínimo. Há duas implementações:

- `DinicMaxFlow`, o algoritmo de Dinitz, usado pelo programa. A BFS preenche um vetor de níveis. A DFS é iterativa, com pilha explícita, para não esgotar a pilha de chamadas em caminhos longos, e mantém para cada vértice a posição da próxima aresta a examinar, de modo que arestas esgotadas não são revisitadas na mesma fase.
- `EdmondsKarpMaxFlow`, mais simples e mais lento, usado como referência nos testes.

Ao final, uma busca a partir da fonte sobre as arestas com capacidade residual marca os vértices do lado S. Esses vértices são os pixels do objeto.

4.4 Decisões de projeto

O corte é calculado sobre uma cópia da foto com no máximo 400 pixels no lado maior. A máscara resultante é ampliada de volta ao tamanho original e a sua borda é suavizada. Com isso, o tempo de resposta fica abaixo de um segundo, o que permite ajustar os parâmetros de forma interativa.

Os valores padrão dos parâmetros, $\lambda = 200$ e $\sigma = 40$, foram escolhidos por experimentação nas fotos de exemplo.

4.5 Testes

O projeto tem 32 testes automáticos em JUnit. Os principais verificam que:

1. a rede de exemplo de Cormen et al. (2012) tem fluxo máximo 23 nas duas implementações;
2. Dinitz e Edmonds-Karp produzem o mesmo valor em 200 redes geradas ao acaso;
3. uma imagem sintética é recortada no contorno esperado;
4. a linha de comando aceita os parâmetros e informa os erros de uso;
5. as regras de dependência entre pacotes são respeitadas.

4.6 Uso

O programa é iniciado pelo comando `thanos`. Sem argumentos, ele apresenta a ajuda:

```
$ ./thanos
Usage: thanos [-hV] <command>
One snap and the background is gone.

Removes the background of a photo by finding the minimum cut
of a graph where every pixel is a vertex.

Options:
  -h, --help      Show this help message and exit.
  -V, --version   Print version information and exit.

Commands:
  cut  Cut the subject out of a photo and save it as a transparent PNG.
  ui   Open the window: scribble, snap, refine, export.
```

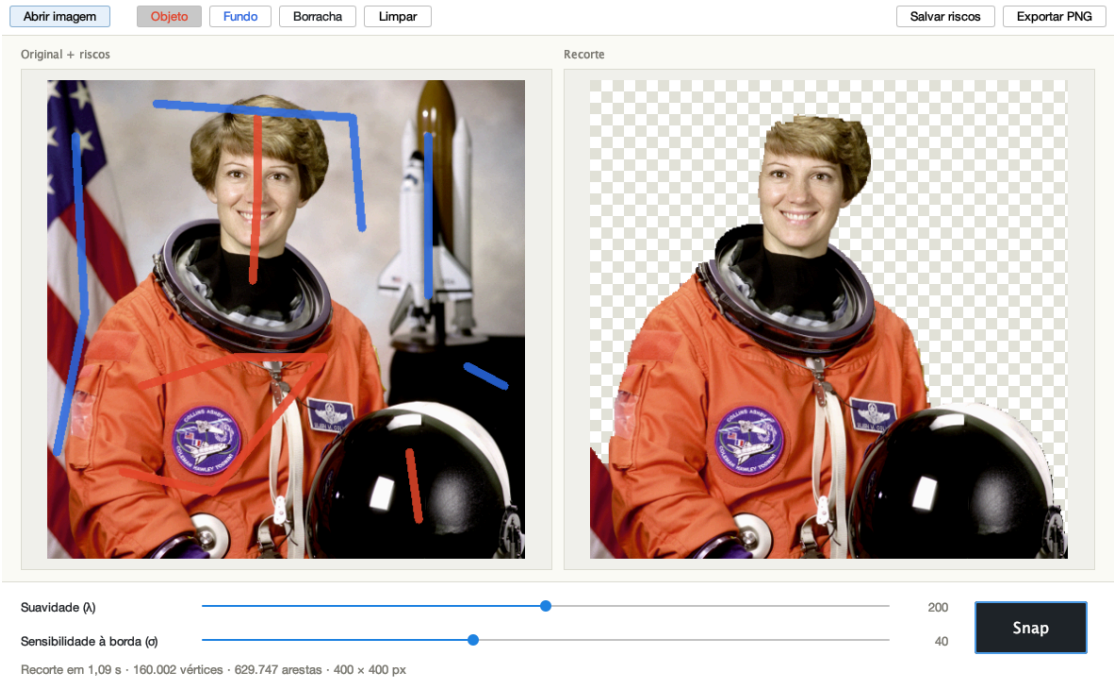
O comando `cut` recorta uma foto pelo terminal. Como não há como desenhar no terminal, os riscos são fornecidos em um arquivo de imagem, com vermelho sobre o objeto e azul sobre o fundo. Ao terminar, o programa informa o tamanho do grafo, o valor do corte mínimo e o tempo:

```
$ ./thanos cut samples/thanos/photo.jpg --scribbles samples/thanos/scribbles.png
Photo             665 x 1000 px
Cut on            266 x 400 px
Graph             106,402 vertices, 416,823 edges
Smoothness        200.0
Edge sensitivity  40.0
Minimum cut       57,511,818
Subject           48.2% of the photo
Time              0.37 s
Saved             samples/thanos/photo-cutout.png
```

Os dois parâmetros do modelo são opções do comando, cada uma com dois nomes: `--lambda` ou `--smoothness` para λ, e `--sigma` ou `--edge-sensitivity` para σ.

O comando `ui` abre a janela mostrada na Figura 7. O usuário abre uma foto, risca o objeto e o fundo com o mouse e aciona o botão Snap. O recorte aparece ao lado, e os dois controles deslizantes recalculam o corte a cada ajuste.

Figura 7 - Janela da aplicação: riscos à esquerda e recorte à direita



Fonte: Os autores (2026), sobre foto da NASA, em domínio público.

5 Resultados

5.1 Desempenho

A Tabela 3 apresenta o tamanho do grafo e o tempo de corte para as cinco fotos de exemplo do repositório, com os parâmetros padrão. As medidas foram feitas em um notebook, com Java 25.

Tabela 3 - Tamanho do grafo e tempo de corte nas fotos de exemplo

Foto	Original	Cópia de trabalho	Vértices	Arestas	Tempo
Astronauta	512 × 512	400 × 400	160.002	626.648	0,96 s
Xícara de café	600 × 400	400 × 266	106.402	416.145	0,30 s
Filhote	600 × 400	400 × 266	106.402	417.514	0,26 s
Pug	600 × 400	400 × 266	106.402	417.218	0,34 s
Thanos	665 × 1000	266 × 400	106.402	416.823	0,37 s

Fonte: Os autores (2026).

O número de vértices é o de pixels da cópia de trabalho mais os dois terminais. O número de arestas fica um pouco abaixo de quatro por pixel porque os pixels das bordas da imagem têm menos vizinhos e os pixels riscados têm apenas uma aresta terminal.

Em todos os casos o corte foi encontrado em menos de um segundo, muito abaixo do que o pior caso $O(V^2E)$ sugere. Esse comportamento é esperado em grafos de imagens, que são regulares e rasos (Boykov; Kolmogorov, 2004).

5.2 Efeito dos parâmetros

A Figura 8 mostra o recorte da mesma foto, com os mesmos riscos, variando um parâmetro de cada vez.

Com $\lambda = 0$ as arestas entre vizinhos não pesam, e cada pixel é decidido só pela cor: as partes da armadura com cor de fundo desaparecem. Com λ alto, separar vizinhos fica caro e o recorte se fecha. Com σ muito baixo, qualquer variação de cor é tratada como borda, e o corte passa por dentro do objeto. Com σ muito alto, quase nenhuma variação é tratada como borda, e o corte deixa de acompanhar o contorno, levando partes do fundo.

Figura 8 - Recortes com valores extremos de λ e de σ



$\lambda = 0$ $\lambda = 400$ $\sigma = 5$ $\sigma = 100$

Fonte: Os autores (2026).

Os valores intermediários adotados como padrão produziram recortes corretos nas cinco fotos, sem ajuste por foto.

6 Conclusão

O trabalho mostrou que um problema de processamento de imagens pode ser resolvido inteiramente com os conceitos da disciplina. A foto foi modelada como um grafo ponderado, a separação entre objeto e fundo foi formulada como um corte mínimo, e o corte foi calculado pelo algoritmo de Dinitz, construído sobre as buscas em largura e em profundidade. O programa recorta fotos de cerca de cem mil pixels em menos de um segundo.

A solução tem três limitações, todas decorrentes do modelo:

1. a borda é rígida, pois cada pixel pertence inteiramente ao objeto ou ao fundo, de modo que fios de cabelo e pelos se perdem;
2. quando o objeto e o fundo têm cores parecidas, a estimativa por cor falha e são necessários mais riscos;
3. o resultado depende de dois parâmetros que podem exigir ajuste conforme a foto.

A terceira limitação explica por que os produtos comerciais adotam redes neurais: elas dispensam os riscos e os parâmetros, ao custo de um treinamento com milhões de imagens e de não ser possível explicar a decisão tomada para cada pixel. O corte em grafo vai no sentido oposto. Ele exige uma indicação do usuário, mas não precisa de treinamento, e cada pixel do resultado pode ser justificado pelas arestas que o corte atravessou.

Por fim, a separação entre o motor e as interfaces permitiu testar o algoritmo de fluxo em redes pequenas, conferidas à mão, antes de aplicá-lo a imagens. O mesmo pacote de grafos serviria, sem alteração, para outros problemas de fluxo, como a distribuição de tarefas entre pessoas.

Referências

BOYKOV, Yuri Y.; JOLLY, Marie-Pierre. Interactive graph cuts for optimal boundary and region segmentation of objects in N-D images. **Proceedings of the Eighth IEEE International Conference on Computer Vision (ICCV)**, v. 1, p. 105–112, 2001.

BOYKOV, Yuri; KOLMOGOROV, Vladimir. An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 26, n. 9, p. 1124–1137, 2004.

CORMEN, Thomas H. *et al.* **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DINIC, E. A. Algorithm for solution of a problem of maximum flow in networks with power estimation. **Soviet Mathematics Doklady**, v. 11, p. 1277–1280, 1970.

DINITZ, Yefim. Dinitz' algorithm: the original version and Even's version. **Lecture Notes in Computer Science**, v. 3895, p. 218–240, 2006.

EDMONDS, Jack; KARP, Richard M. Theoretical improvements in algorithmic efficiency for network flow problems. **Journal of the ACM**, v. 19, n. 2, p. 248–264, 1972.

FORD, L. R.; FULKERSON, D. R. Maximal flow through a network. **Canadian Journal of Mathematics**, v. 8, p. 399–404, 1956.

KLEINBERG, Jon; TARDOS, Éva. **Algorithm Design**. Boston: Pearson Addison-Wesley, 2006.

MONTEIRO, Ruy Diniz; INGHAM, Amanda Bialer. **Thanos: remoção de fundo de imagens por corte mínimo em grafos**. Disponível em: <https://github.com/ruymon/thanos>. Acesso em: 12 nov. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.